

Accepting User Input in Java

Using Command Line Arguments and Scanner Class

Module Objective

After completing this module, students will be able to:

- Understand different ways of accepting input in Java.
 - Use **Command Line Arguments** to pass values during program execution.
 - Use the **Scanner** class to accept input from the keyboard.
 - Differentiate between `next()`, `nextLine()`, `nextInt()`, `nextDouble()`, etc.
 - Convert String values into primitive data types.
 - Identify common mistakes while using the Scanner class.
-

Introduction

Most Java applications require input from users. For example,

- A banking application asks for an account number.
- A shopping application asks for the quantity of items.
- A student management system asks for the student's name and marks.

Java provides multiple ways to accept input. The two most commonly used methods are:

1. **Command Line Arguments**
 2. **Scanner Class**
-

Method 1: Command Line Arguments

What are Command Line Arguments?

Command Line Arguments are the values passed to a Java program **at the time of execution**.

These values are automatically stored inside the **String array (`args`)** of the `main()` method.

```
public static void main(String[] args)
```

Here,

- `String[]` represents an array of Strings.
 - `args` stores all the values supplied through the command line.
-

Real-Life Analogy

Imagine ordering a pizza over the phone.

Before the restaurant starts preparing your order, you tell them:

- Pizza Type
- Size
- Quantity

These details are provided **before** the order is processed.

Similarly, Command Line Arguments are supplied **before** the Java program starts executing.

Syntax

```
java ClassName value1 value2 value3
```

Example

```
java Demo Rahul 25 Delhi
```

The array contains:

```
args[0] = Rahul  
args[1] = 25  
args[2] = Delhi
```

Example Program

```
public class Demo {  
  
    public static void main(String[] args) {  
  
        System.out.println(args[0]);  
        System.out.println(args[1]);  
    }  
}
```

```
        System.out.println(args[2]);  
    }  
}
```

Output

```
Rahul  
25  
Delhi
```

Important Point

Every command line argument is stored as a **String**.

Example

```
java Demo 100
```

Java stores

```
"100"
```

and not

```
100
```

Checking the Data Type

```
System.out.println(args[0] instanceof String);
```

Output

```
true
```

String Concatenation

```
System.out.println(args[0] + 20);
```

Suppose

```
args[0] = "50"
```

Output

```
5020
```

Why?

Because Java treats "50" as a String.

Converting String into Integer

To perform arithmetic operations,

use

```
Integer.parseInt()
```

Example

```
int number = Integer.parseInt(args[0]);  
System.out.println(number + 20);
```

Output

70

Other Conversion Methods

Method	Converts To
Integer.parseInt()	int
Double.parseDouble()	double
Float.parseFloat()	float
Long.parseLong()	long
Boolean.parseBoolean()	boolean

Advantages of Command Line Arguments

- Easy to use
 - Suitable for batch processing
 - Useful in automation
 - No additional classes required
-

Limitations

- All inputs are Strings.
 - User cannot enter values after program execution begins.
 - Not suitable for interactive applications.
-

Method 2: Scanner Class

What is Scanner?

Scanner is a predefined Java class that allows users to enter data using the keyboard during program execution.

Package

```
import java.util.Scanner;
```

Creating Scanner Object

```
Scanner scanner = new Scanner(System.in);
```

Here,

```
System.in
```

represents the standard keyboard input.

Real-Life Analogy

Suppose a teacher asks questions one by one.

Student answers immediately.

Similarly,

Scanner accepts values while the program is running.

Reading Integer

```
int age = scanner.nextInt();
```

Reading Decimal Number

```
double salary = scanner.nextDouble();
```

Reading Float

```
float percentage = scanner.nextFloat();
```

Reading Long

```
long mobileNumber = scanner.nextLong();
```

Reading Boolean

```
boolean status = scanner.nextBoolean();
```

Reading a Single Word

```
String name = scanner.next();
```

Input

```
Rahul Kumar
```

Output

```
Rahul
```

Only the first word is read.

Reading an Entire Line

```
String name = scanner.nextLine();
```

Input

```
Rahul Kumar
```

Output

Difference between next() and nextLine()

next()	nextLine()
Reads one word	Reads complete sentence
Stops at space	Reads until Enter key
Ignores remaining words	Reads the complete line

Example

Input

Rahul Kumar Sharma

Using

```
scanner.next();
```

Output

Rahul

Using

```
scanner.nextLine();
```

Output

Rahul Kumar Sharma

Common Problem

```
Scanner scanner = new Scanner(System.in);
```

```
int age = scanner.nextInt();
```

```
String name = scanner.nextLine();
```

```
System.out.println(name);
```

Output

Age : 20

Name :

Name appears blank.

Why Does This Happen?

`nextInt()` reads only

20

It leaves the Enter key (`\n`) in the input buffer.

Immediately,

`nextLine()`

reads that newline instead of waiting for user input.

Correct Solution

```
int age = scanner.nextInt();
```

```
scanner.nextLine();
```

```
String name = scanner.nextLine();
```

Now the program works correctly.

Sample Program Using Scanner

```
Scanner scanner = new Scanner(System.in);
```

```
System.out.print("Enter Name : ");
```

```
String name = scanner.nextLine();
```

```
System.out.print("Enter Age : ");
```

```
int age = scanner.nextInt();
```

```
System.out.print("Enter Marks : ");
```

```
double marks = scanner.nextDouble();
```

```
System.out.println("Name : " + name);
```

```
System.out.println("Age : " + age);
System.out.println("Marks : " + marks);

scanner.close();
```

Output

```
Enter Name : Rahul Sharma
Enter Age : 20
Enter Marks : 85.5
```

```
Name : Rahul Sharma
Age : 20
Marks : 85.5
```

Scanner Methods

Method	Purpose
next()	Reads one word
nextLine()	Reads complete line
nextInt()	Reads integer
nextDouble()	Reads decimal number
nextFloat()	Reads float
nextBoolean()	Reads true/false
nextLong()	Reads long value
nextShort()	Reads short value
nextByte()	Reads byte value

Command Line Arguments vs Scanner

Feature	Command Line Arguments	Scanner Class
Input Time	Before execution	During execution
Input Source	Command Prompt	Keyboard
Data Type	Always String	Supports multiple data types
User Interaction	No	Yes
Type Conversion	Required	Not required for primitive types
Best Used For	Automation, scripts	Interactive applications

Best Practices

- Always validate the number of command line arguments before accessing them.
- Use meaningful variable names.
- Close the Scanner object using `scanner.close()` after use.

- Use `nextLine()` carefully after `nextInt()`, `nextDouble()`, etc.
 - Convert Strings into numeric types only when necessary.
-

Summary

- **Command Line Arguments** are used to pass input **before** the program starts and are always received as **Strings**.
- The **Scanner** class allows programs to accept user input **during execution** and supports multiple data types such as `int`, `double`, `String`, and `boolean`.
- `next()` reads a single word, while `nextLine()` reads an entire line.
- When using `nextInt()` (or similar methods) followed by `nextLine()`, consume the leftover newline to avoid unexpected behavior.
- Choosing between Command Line Arguments and the Scanner class depends on the application's requirements—automation versus interactive input. This knowledge forms the foundation for developing Java applications that interact effectively with users.

Java program that demonstrates the two most common ways of accepting input in Java:

1. **Command Line Arguments**
2. **Scanner Class**

The code is separated into two methods for better understanding.

Java Program: Accepting User Input

```
package com.company.basic;

import java.util.Scanner;

/**
 * Program to demonstrate different ways of accepting input in
 * Java.
 * 1. Command Line Arguments
 * 2. Scanner Class
 *
 * Author : Your Name
 */

public class InputDemo {

    public static void main(String[] args) {

        System.out.println("===== COMMAND LINE ARGUMENTS
        =====");
        commandLineInput(args);

        System.out.println("\n===== SCANNER CLASS
        =====");
        scannerInput();
    }

    /**
     * Demonstrates input using Command Line Arguments
     */
    public static void commandLineInput(String[] args) {

        if (args.length < 3) {
            System.out.println("Please provide three command
            line arguments.");
            System.out.println("Example:");
            System.out.println("java InputDemo 10 Rahul 85.5");
        }
    }
}
```

```

        return;
    }

    // Reading command line arguments
    String age = args[0];
    String name = args[1];
    String marks = args[2];

    System.out.println("Age      : " + age);
    System.out.println("Name     : " + name);
    System.out.println("Marks    : " + marks);

    // Every command line argument is a String
    System.out.println("\nAll command line arguments are
String objects.");

    System.out.println(age instanceof String);
    System.out.println(name instanceof String);
    System.out.println(marks instanceof String);

    // String Concatenation
    System.out.println("\nString Concatenation");
    System.out.println(age + 10);

    // Type Conversion
    int convertedAge = Integer.parseInt(age);

    System.out.println("After Integer Conversion");
    System.out.println(convertedAge + 10);
}

/**
 * Demonstrates input using Scanner Class
 */
public static void scannerInput() {

    Scanner scanner = new Scanner(System.in);

    System.out.print("Enter Student Name : ");
    String name = scanner.nextLine();

    System.out.print("Enter Age : ");
    int age = scanner.nextInt();

    System.out.print("Enter Marks : ");
    double marks = scanner.nextDouble();

```

```
        System.out.println("\n----- Student Details -----");
        System.out.println("Name   : " + name);
        System.out.println("Age    : " + age);
        System.out.println("Marks  : " + marks);

        scanner.close();
    }
}
```

Concept 1: Command Line Arguments

What are Command Line Arguments?

Command Line Arguments are **values supplied to the program at the time of execution.**

They are automatically stored in the **String[] args** parameter of the `main()` method.

```
public static void main(String[] args)
```

Here,

- `String[]` → Array of Strings
 - `args` → Stores all the command line arguments
-

Example

Program:

```
public class Demo {  
    public static void main(String[] args) {  
        System.out.println(args[0]);  
        System.out.println(args[1]);  
    }  
}
```

Execution

```
java Demo Rahul 25
```

Output

```
Rahul  
25
```

Important Point

Even if you pass numbers,

```
java Demo 100
```

Java stores it as

"100"

which is a **String**.

Therefore,

```
System.out.println(args[0] + 10);
```

Output

10010

because it performs **String concatenation**.

To perform arithmetic operations,

convert the String into an integer.

```
int number = Integer.parseInt(args[0]);  
System.out.println(number + 10);
```

Output

110

Common Conversion Methods

Method	Purpose
Integer.parseInt()	Converts String to int
Double.parseDouble()	Converts String to double
Float.parseFloat()	Converts String to float
Long.parseLong()	Converts String to long
Boolean.parseBoolean()	Converts String to boolean

Advantages

- Fast
 - Useful for automation
 - Useful in batch processing
 - Useful while executing programs through terminal
-

Limitations

- Every value is stored as String
 - Not user-friendly
 - Cannot ask input after execution starts
-

Concept 2: Scanner Class

The Scanner class allows the user to enter input **while the program is running**.

Package

```
import java.util.Scanner;
```

Creating Scanner Object

```
Scanner scanner = new Scanner(System.in);
```

Reading Different Types of Input

Integer

```
int age = scanner.nextInt();
```

Double

```
double salary = scanner.nextDouble();
```

String (Single Word)

```
String name = scanner.next();
```

Input

Rahul Kumar

Output

Rahul

Only first word is read.

String (Complete Line)

```
String name = scanner.nextLine();
```

Input

Rahul Kumar

Output

Rahul Kumar

Why is `nextLine()` Required after `nextInt()` ?

Example

```
int age = scanner.nextInt();  
String name = scanner.nextLine();
```

Output

Age : 21

Name :

Name becomes empty.

Reason:

`nextInt()`

reads only

21

The Enter key (`\n`) remains in the input buffer.

Immediately,

`nextLine()`

reads that leftover newline.

Correct Way

```
int age = scanner.nextInt();

scanner.nextLine(); // Consume newline

String name = scanner.nextLine();
```

Now,

```
Age : 21
Name : Rahul Sharma
```

works correctly.

Comparison: Command Line Arguments vs Scanner

Feature	Command Line Arguments	Scanner Class
Input Time	Before program starts	During program execution
Source	Terminal/Command Prompt	Keyboard
Data Type	Always String	Supports int, double, String, etc.
Type Conversion	Required	Not required for primitive types
User Interaction	No	Yes
Best Use	Automation, scripts, batch jobs	Interactive applications

Sample Output

Command Line Execution

```
java InputDemo 21 Rahul 92.5
===== COMMAND LINE ARGUMENTS =====
Age      : 21
Name     : Rahul
Marks    : 92.5
```

```
All command line arguments are String objects.
true
true
true
```

String Concatenation

2110

After Integer Conversion

31

===== SCANNER CLASS =====

Enter Student Name : Rahul Sharma

Enter Age : 21

Enter Marks : 92.5

----- Student Details -----

Name : Rahul Sharma

Age : 21

Marks : 92.5

This version is ideal for classroom teaching because it clearly separates the two input techniques, demonstrates type conversion, highlights common pitfalls (such as `nextInt()` followed by `nextLine()`), and follows Java coding standards.